DATA 583

Advanced Predictive Modelling

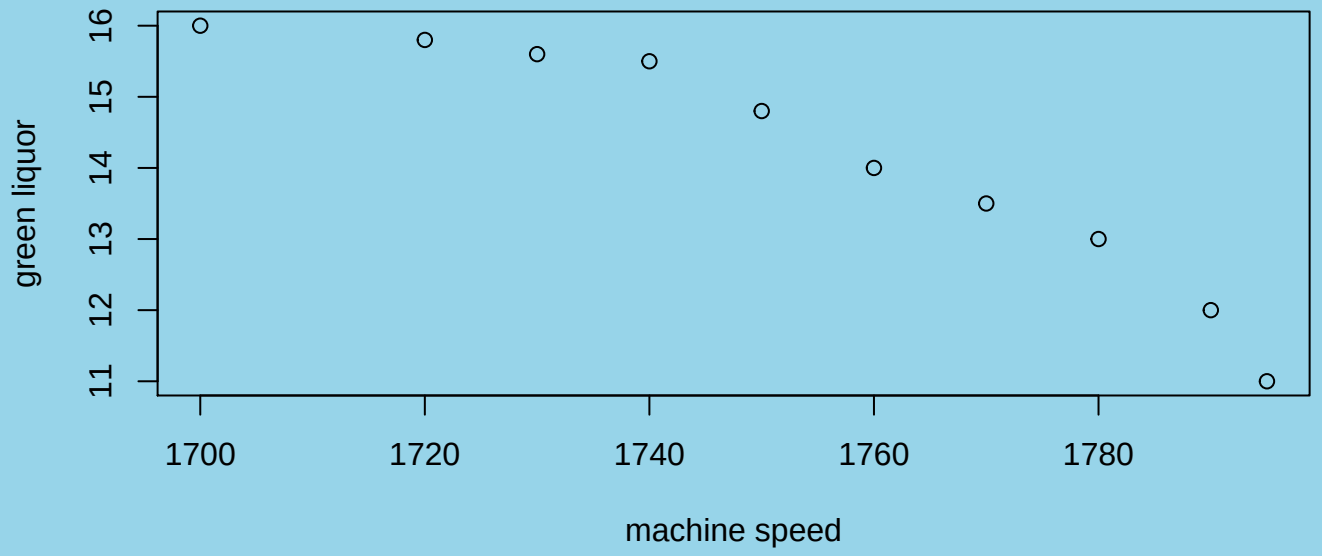
Splines for Regression and Smoothing

- **Preliminary examples of data where linear models are not appropriate**
- **Polynomial models – review of multiple regression; design matrix**
- **Spline models - some theory, design matrix; fitting with `lm`**
- **Smoothing Splines**

Reference: Wood, S. (2017) *Generalized Additive Models*. 2nd Edition. Boca Raton: Chapman & Hall/CRC.

Preliminary examples - data from a paper mIII

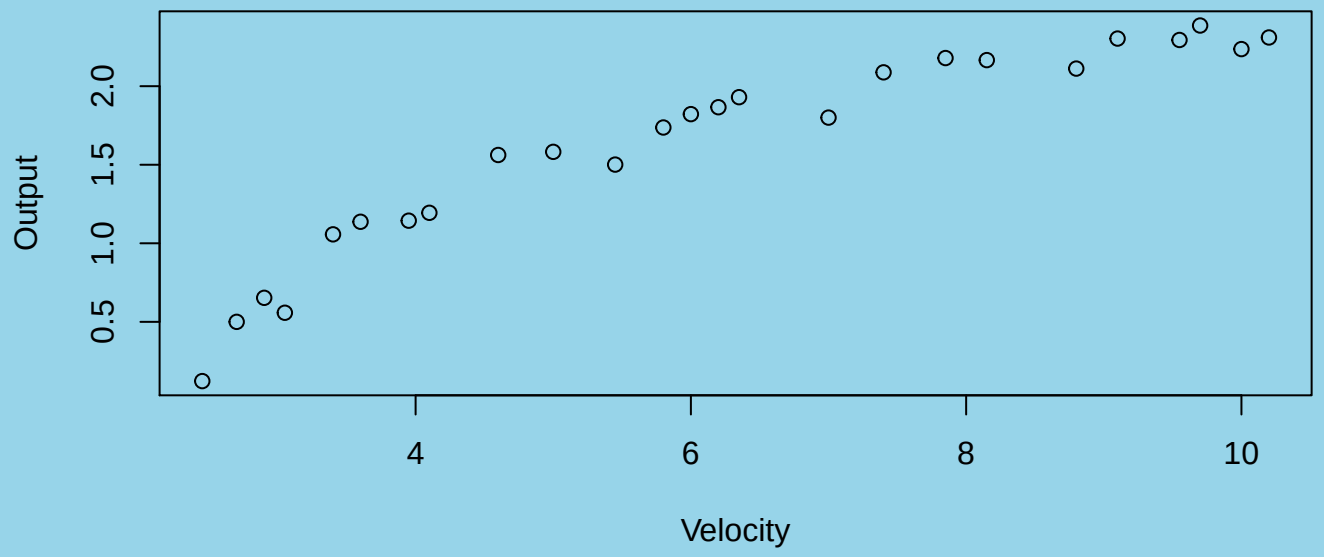
```
library(MPV)
plot(y ~ x, ylab = "green liquor", xlab = "machine speed",
     data = p7.19)
```



Measurements of the amount of green liquor produced are recorded for various speeds. How does the speed of a paper mill machine affect quality of the finished product?

Examples - Data from a Windmill Electric Turbine

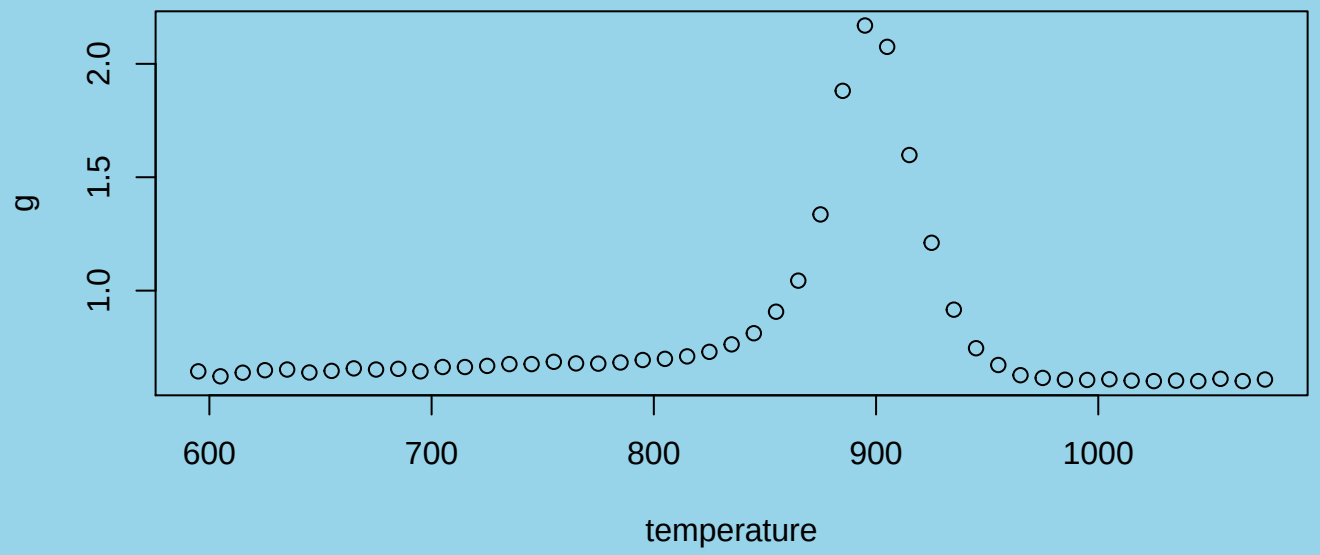
```
source("windmill.R")
plot(Output ~ Velocity, data = windmill)
```



These data are measurements of electrical output for a windmill turbine at various windspeeds.

Preliminary examples - data from a standards test

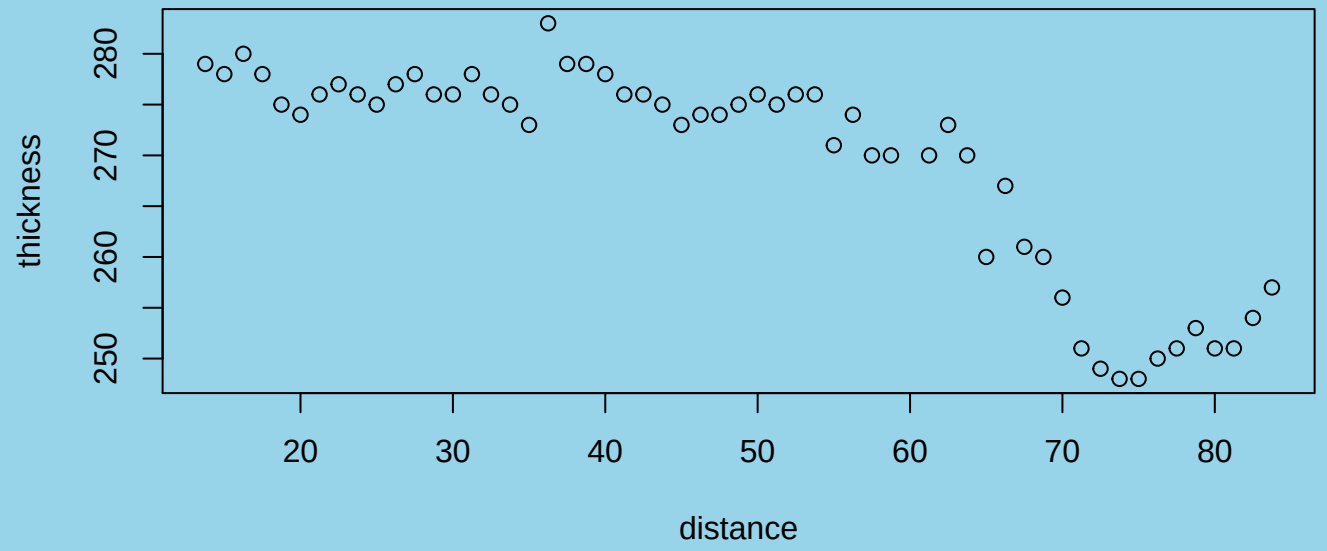
```
source("titanium.R")
plot(g ~ temperature, data = titanium)
```



These data are measurements of a certain thermal property of titanium as a function of temperature (presumably in Kelvin degrees).

Preliminary examples - data from the oil fields

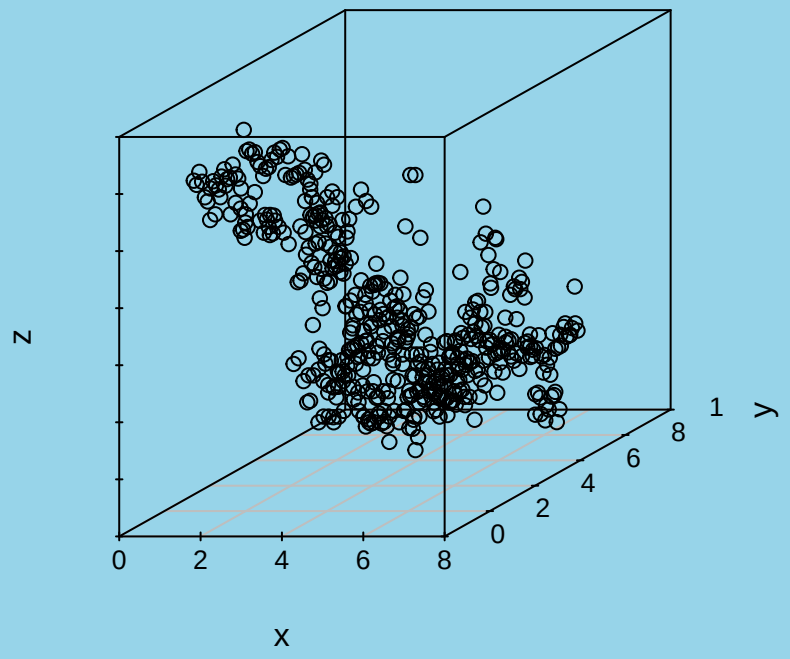
```
library(DAAG)
plot(thickness ~ distance, data = geophones)
```

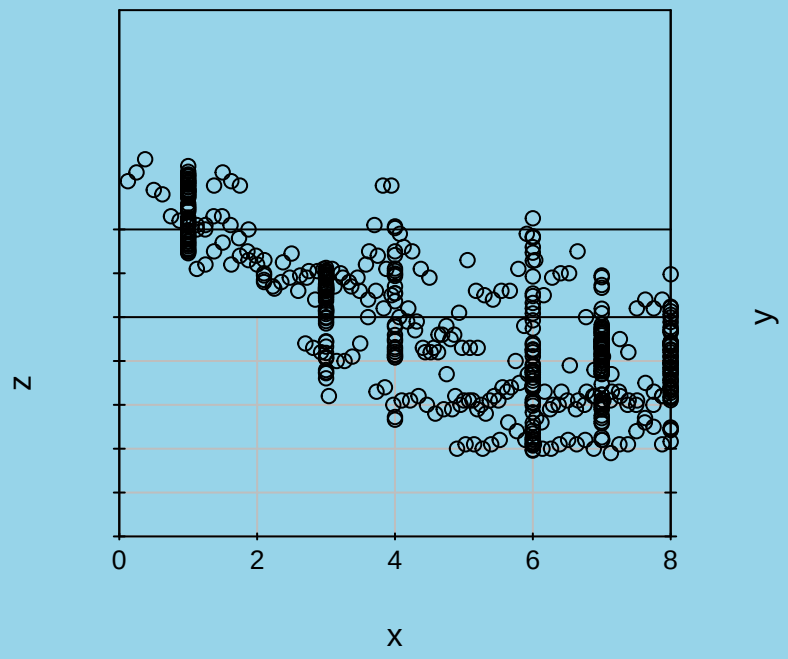


These data refer to a subterranean geological formation in Central Alberta; the measurements were recorded by a network of geophones, and are subject to error.

Preliminary examples - larger set of seismic timings

```
source("seismictimingsfull.R");library("scatterplot3d");
par(mfrow=c(1,2))
with(seismictimingsfull, scatterplot3d(x, y, z, z.ticklabs="",
    mar = c(5, 3, 0, 3) + 0.1))
with(seismictimingsfull, scatterplot3d(x, y, z, angle=90,
z.ticklabs="", y.ticklabs="", mar = c(5, 3, 0, 3) + 0.1))
```





Modelling these data sets

- In all cases, linear regression, multiple or otherwise, is not going to give accurate predictions.

- Alternatives:
  - Polynomial regression
  - Smoothing
    - Splines
    - Penalized and Smoothing Splines

Quadratic regression

Quadratic model:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \varepsilon_i$$

ε_i are assumed to be uncorrelated with mean 0 and constant variance σ^2.

In matrix form:

$$\mathbf{y} = \mathbf{X}\beta + \epsilon$$

with

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \quad \epsilon = \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix} \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

$\rightsquigarrow$ **Regression of y on x and x^2. Estimation, testing with `lm()` applies.**

Example - paper mill data

```
paper.lm2 <- lm(y ~ x + I(x^2), data = p7.19)
summary(paper.lm2)

##
## Call:
## lm(formula = y ~ x + I(x^2), data = p7.19)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.27575 -0.06414 -0.03741  0.16051  0.27187
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.709e+03  2.448e+02  -6.984 0.000215 ***
## x            2.023e+00  2.798e-01   7.230 0.000173 ***
## I(x^2)      -5.929e-04  7.994e-05  -7.417 0.000147 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2101 on 7 degrees of freedom
## Multiple R-squared:  0.9885,Adjusted R-squared:  0.9852
## F-statistic: 300.1 on 2 and 7 DF,  p-value: 1.645e-07
```

Example - paper mill data

Fitted model:

$$\widehat{\mu}(x) = -1709 + 2.02x - .00059x^2$$

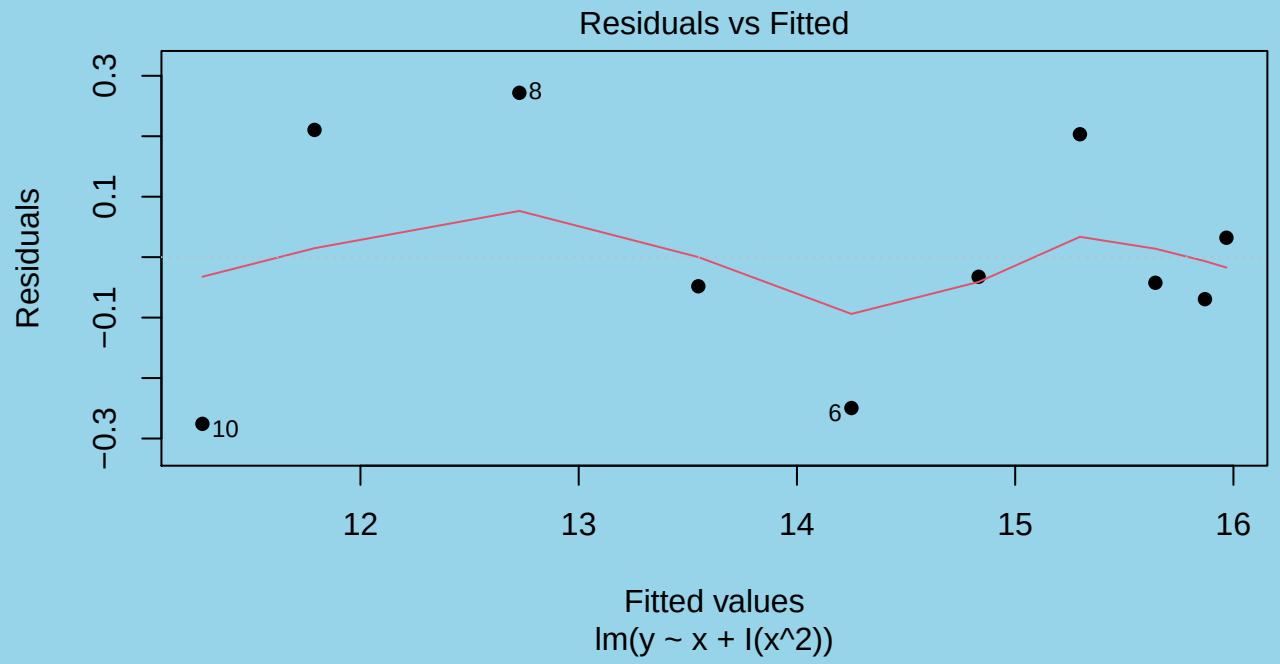
Noise standard deviation estimate = .21.

Is this model satisfactory?

The standard model diagnostics, residual plots and influence measures, can be used.

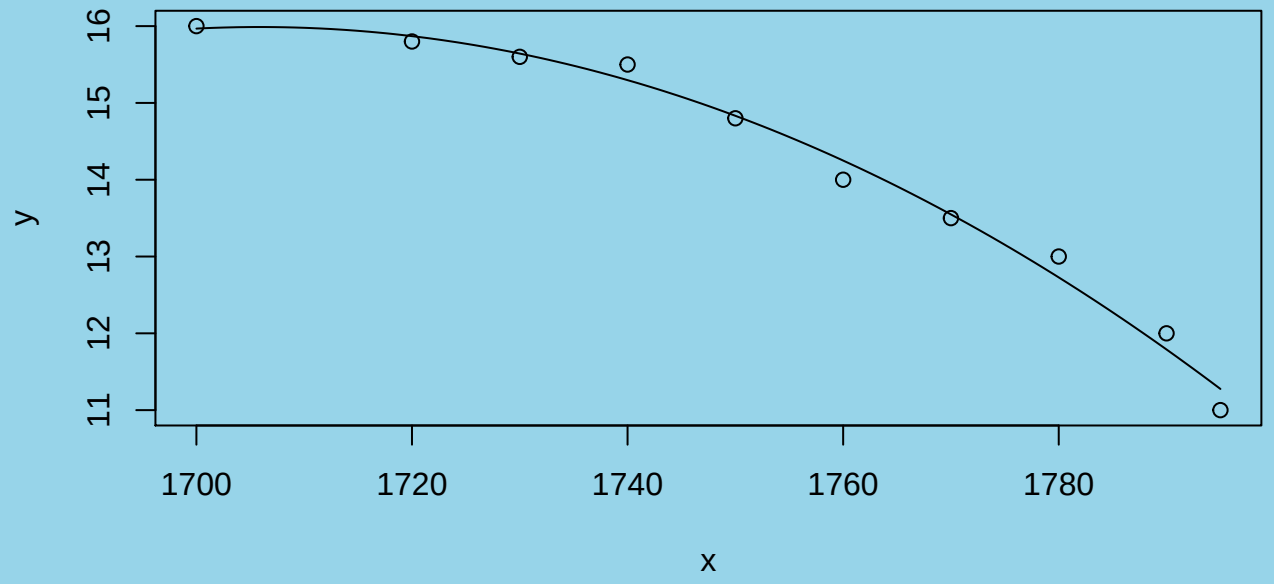
Example - paper mill data

```
plot(paper.lm2, which=1, pch=16)
```



Example - paper mill data

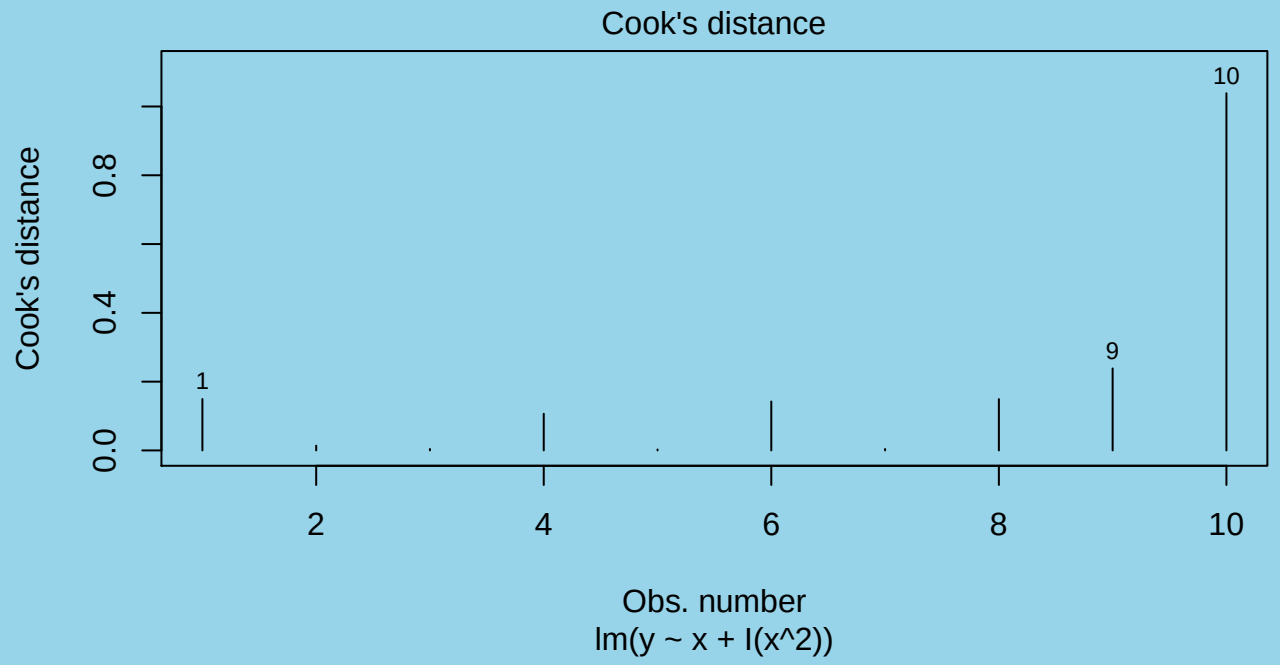
```
plot(y ~ x, data = p7.19)
quadline(paper.lm2)
```



Example - Paper mill data

Check for influential observations:

```
plot(paper.lm2, which=4, pch=16)
```



Example - paper mill data

Check observation 10 more closely:

```
dfbetas(paper.lm2)[10,]

##  (Intercept)             x        I(x^2)
##    -1.383708     1.395116    -1.406660

dffits(paper.lm2)[10]

##         10
## -2.258152
```

Observation 10 is highly influential.

General polynomial regression model

- kth degree polynomial:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_k x^k + \varepsilon$$

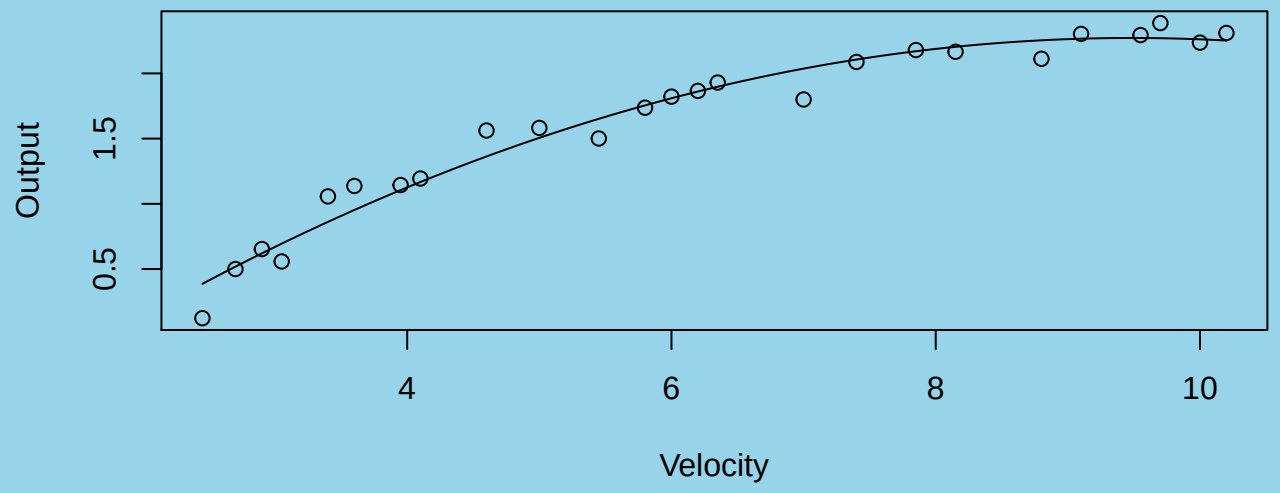
- k is referred to as the 'order', e.g. quadratic model is 2nd order

- View this as a regression of y on the k regressors $x_1 = x, x_2 = x^2, \ldots, x_k = x^k$.

- The design matrix X has $k + 1$ columns containing $1, x, x^2, \ldots, x^k$, evaluated at the values of x.

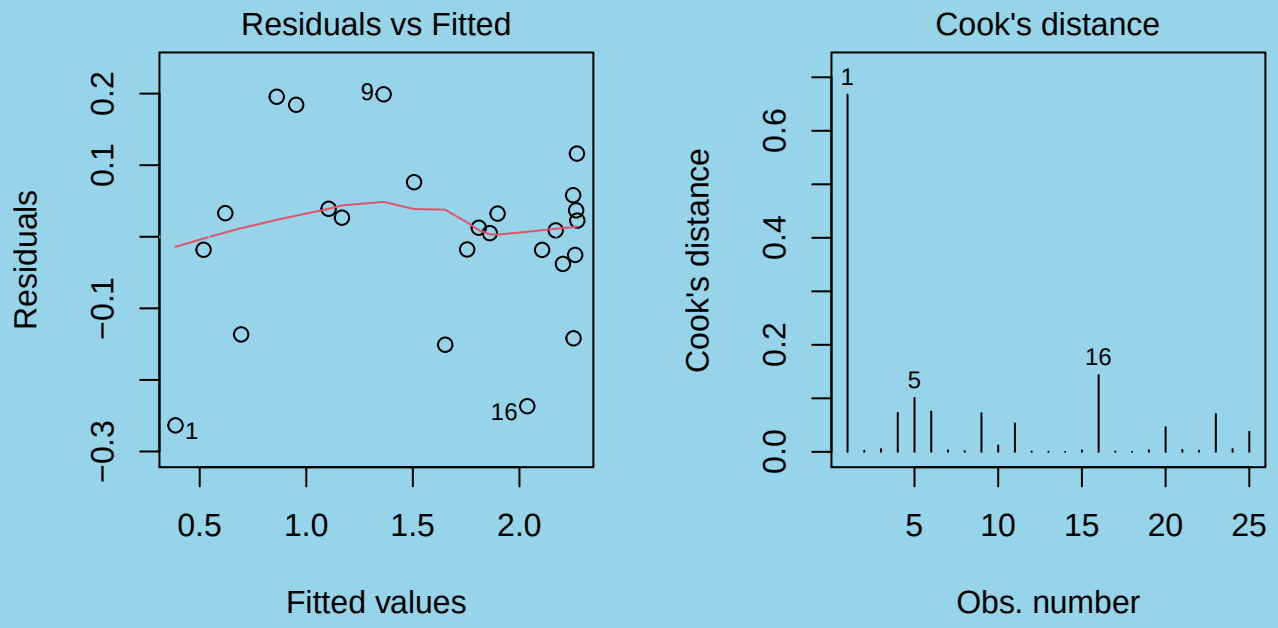
Example – windmill data

```
windmill.lm2 <- lm(Output ~ Velocity + I(Velocity^2), data = windmill)
plot(Output ~ Velocity, data = windmill)
quadline(windmill.lm2)
```



Example – windmill data

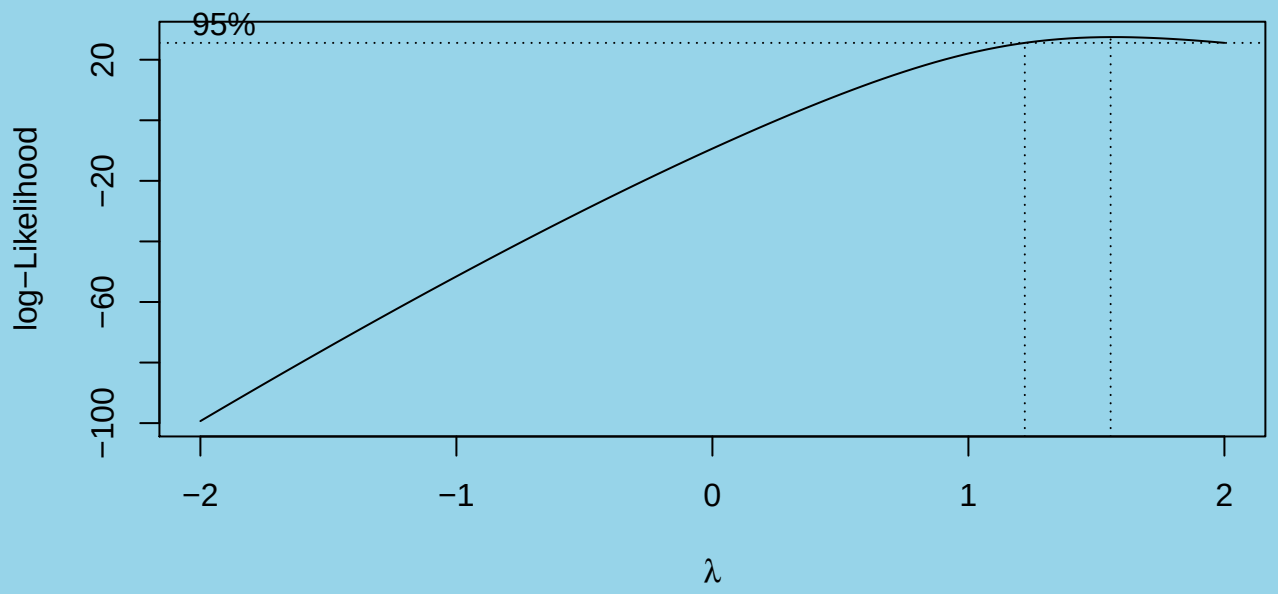
```
par(mfrow=c(1, 2))
plot(windmill.lm2, which=1); plot(windmill.lm2, which = 4)
```



Another influential outlier. And does this model even make sense? What will happen at larger values of velocity?

Transforming the response - Box-Cox transformation

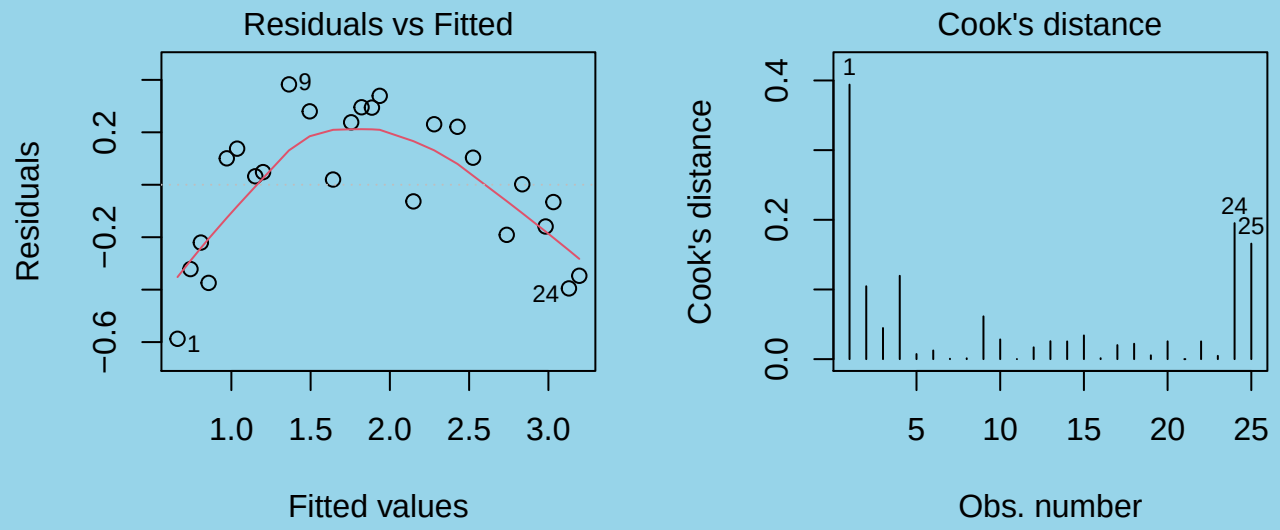
```
library(MASS) # contains the boxcox() function
boxcox(windmill.lm2)
```



This procedure find the mle for the power λ in a model relating y^λ to the predictor. Here, a power near 1.25 is suggested.

Example – windmill data

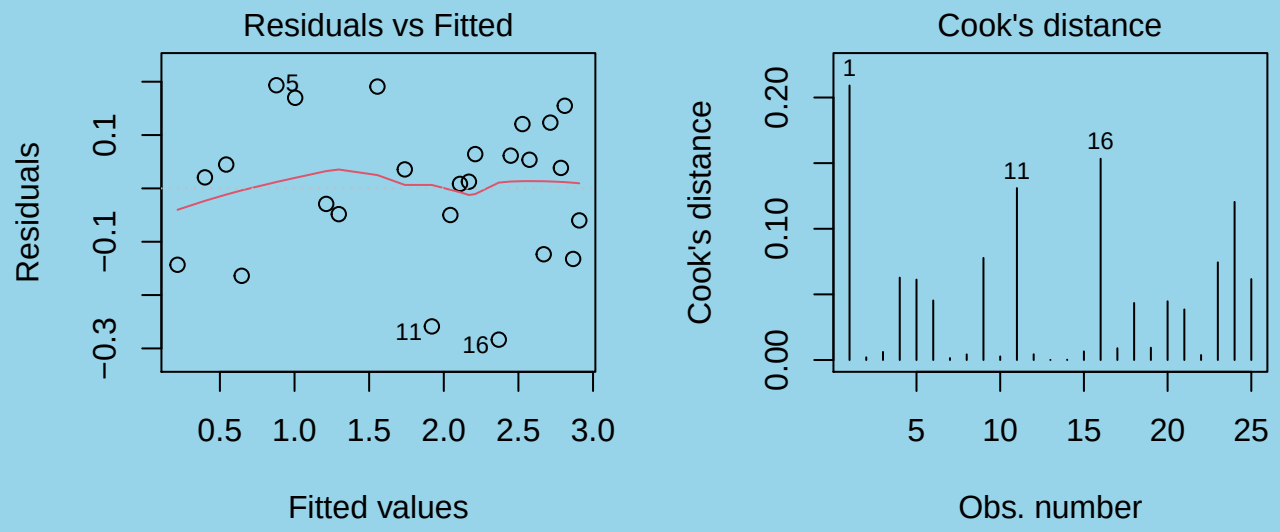
```
windmill$out <- windmill$Output^1.25
windmill.lm <- lm(out ~   Velocity,
      data = windmill)
par(mfrow=c(1, 2))
plot(windmill.lm, which=1); plot(windmill.lm, which = 4)
```



Definitely not a good fit.

Cubic regression for the transformed windmill data

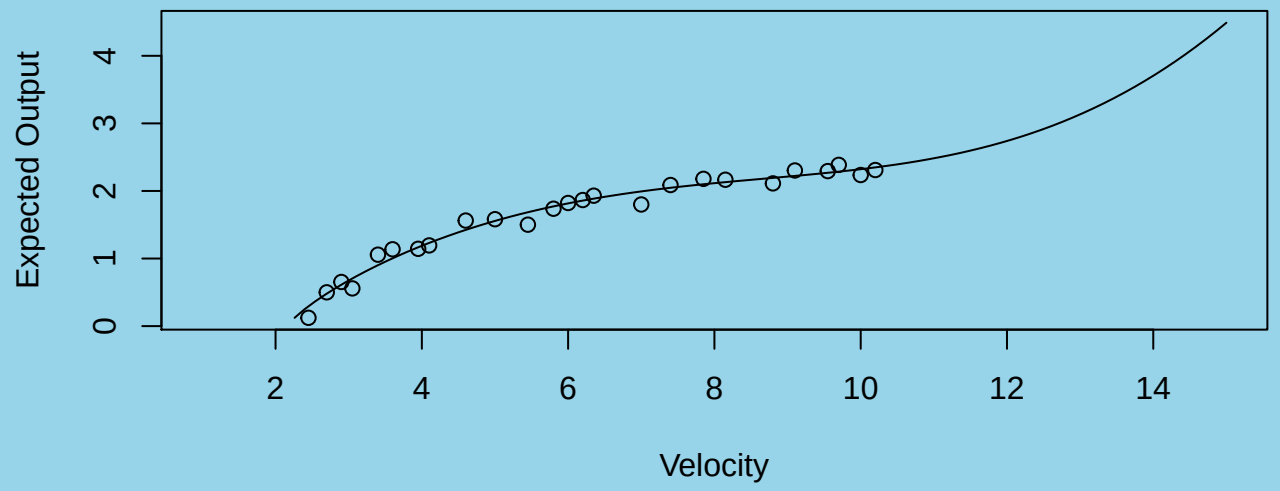
```
windmill$v <- windmill$Velocity^1.25
windmill.lm <- lm(out ~ v + I(v^2) + I(v^3), data = windmill)
par(mfrow=c(1, 2))
plot(windmill.lm, which=1); plot(windmill.lm, which = 4)
```



a good fit ... but ...

Cubic regression for the transformed windmill data

```
x <- seq(1, 15, length=101)
plot(x, (predict(windmill.lm,
     newdata = data.frame(v = x^1.25)))^(1/1.25),
     ylab="Expected Output", xlab="Velocity", type="l")
with(windmill, points(Output ~ Velocity))
```



Extrapolation in either direction does not make sense.

Important considerations when using polynomials

- **Order of the Model**

  Keep this as low as possible – parsimony

- **Use Orthogonal Polynomials if Testing the Order**

  In R, `poly()` transforms $\mathbf{X}$ to an orthogonal matrix, $\mathbf{X}'$, whose columns are $P_0(x), P_1(x), \ldots, P_k(x)$, which are polynomials evaluated at the values of x.

  Model becomes

$$\mathbf{y}' = \mathbf{X}'\beta' + \epsilon$$

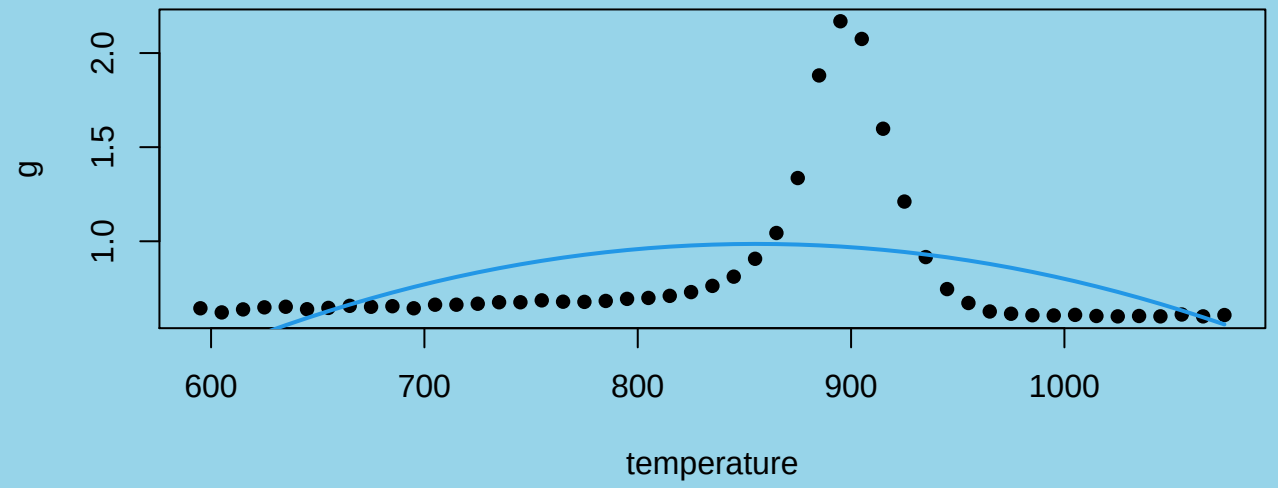
Example – titanium heat data - quadratic model

```
titanium.lm2 <- lm(g~poly(temperature, 2), data = titanium)
summary(titanium.lm2)$coefficients

##                              Estimate Std. Error   t value     Pr(>|t|)
## (Intercept)                 0.8045918 0.04888047 16.460394 6.630450e-21
## poly(temperature, 2)1       0.3605537 0.34216331  1.053748 2.975022e-01
## poly(temperature, 2)2      -1.1114465 0.34216331 -3.248292 2.171648e-03
```

2nd degree term is significant.

Example – titanium heat data - quadratic model

```
plot(titanium, pch=16)
with(titanium, lines(temperature, predict(titanium.lm2), col=4, lwd=2))
```



A pretty miserable fit

Does a higher order polynomial fit better?

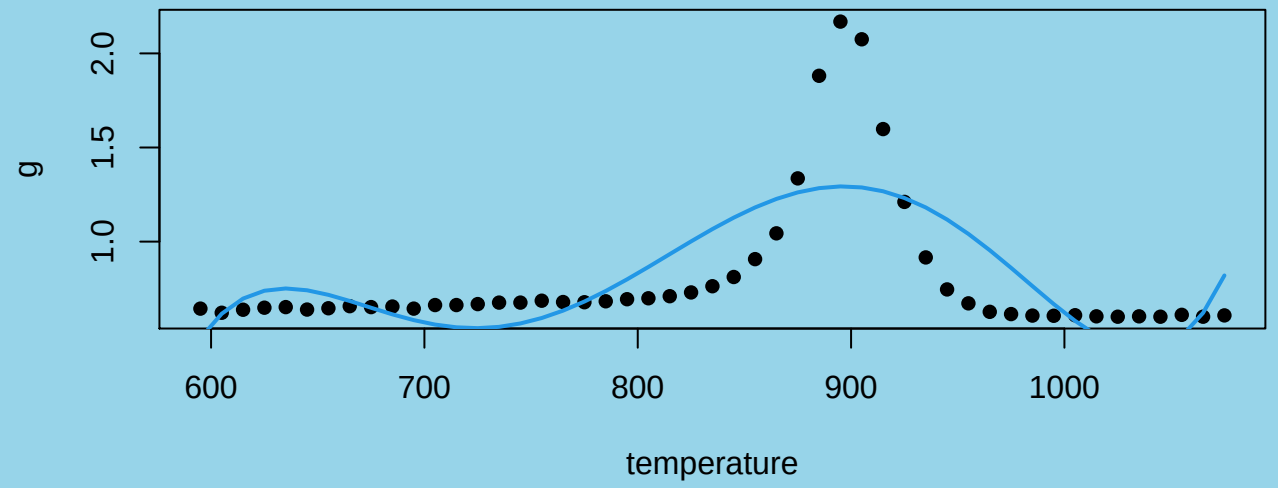
5th degree fit:

```
titanium.lm5 <- lm(g~poly(temperature, 5), data = titanium)
summary(titanium.lm5)$coefficients

##                              Estimate Std. Error   t value     Pr(>|t|)
## (Intercept)                 0.8045918 0.03875567 20.760621 4.927700e-24
## poly(temperature, 5)1  0.3605537 0.27128971  1.329036 1.908454e-01
## poly(temperature, 5)2 -1.1114465 0.27128971 -4.096899 1.817190e-04
## poly(temperature, 5)3 -0.8865013 0.27128971 -3.267729 2.135370e-03
## poly(temperature, 5)4  0.5238606 0.27128971  1.931001 6.009272e-02
## poly(temperature, 5)5  1.0772421 0.27128971  3.970818 2.680479e-04
```

5th degree term is significant.

Example – titanium heat data - 5th Degree Model

```
plot(titanium, pch=16)
with(titanium, lines(temperature, predict(titanium.lm5), col=4, lwd=2))
```



Still a pretty miserable fit

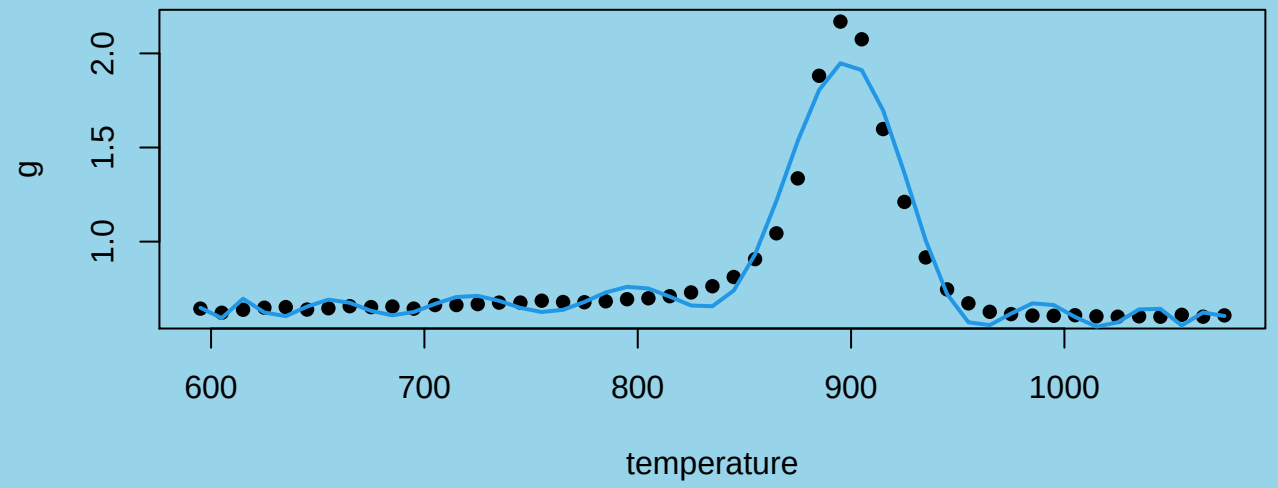
How about a 17 degree model?

```
titanium.lm17 <- lm(g~poly(temperature, 17), data = titanium)
summary(titanium.lm17)$coefficients[18,]

##       Estimate     Std. Error         t value       Pr(>|t|)
## -0.3964265281  0.0950111068 -4.1724230093  0.0002257095
```

17th degree term is significant.

Example – titanium heat data - 17 Degree Model

```
plot(titanium, pch=16)
with(titanium, lines(temperature, predict(titanium.lm17), col=4, lwd=2))
```



Better, but at the cost of 18 estimated parameters ... and still pretty bad.

- Polynomials are not flexible enough to adequately model all smooth functions.
- Piecewise polynomials or splines are more flexible.
- What is a piecewise polynomial?
- Example: Consider two polynomials

$$p_1(x) = -2x^2 + .5x^3$$

$$p_2(x) = -2x^2 + .5x^3 - 2(x-6)^3$$

An example of a piecewise polynomial with 2 pieces

- **We can make a piecewise polynomial out of these two polynomials by cutting them at $x = 6$ (they are equal there) and tying them together with a knot at $x = 6$:**

$$s(x) = -2x^2 + .5x^3, \quad x < 6$$

$$s(x) = -2x^2 + .5x^3 - 2(x-6)^3, \quad x \geq 6$$

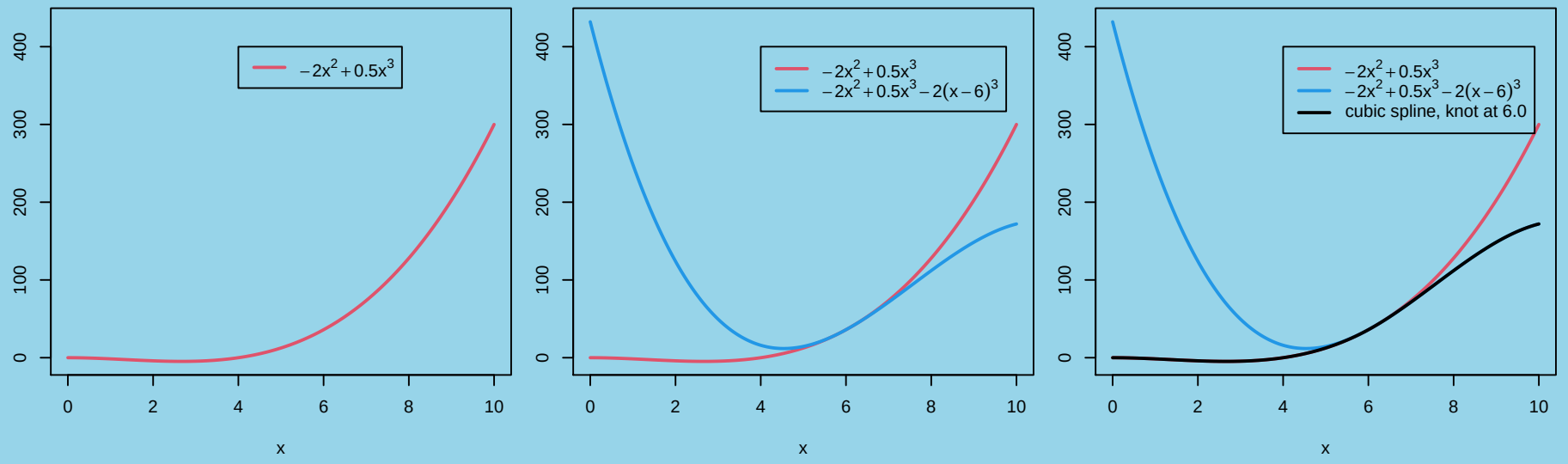
or

$$s(x) = -2x^2 + .5x^3 - 2(x-6)^3 I(x \geq 6)$$

- **This is a cubic spline with a knot at 6.**

An example of a piecewise polynomial with 2 pieces

```
source("splineeg.R")   # some spline examples
par(mfrow=c(1, 3), mar=c(4, 2, 1, 1))
spline.eg()     # plots two polynomials and a spline
```



Truncated power functions

$$T(x) = (x - \tau)_+^k = (x - \tau)^k I(x \geq \tau)$$

$T(x)$ **is 0 for** $x < \tau$ **and** $(x - \tau)^k$ **for** $x \geq \tau$**.**

$T(x)$ **is a** k **degree spline with a knot at** τ**.**

Example:

$$s(x) = -2x^2 + .5x^3 - 2(x - 6)_+^3$$

Truncated power functions

R code for a truncated power function:

```
tr.pwr


## function (x, knot, degree = 3)
## {
##     (x > knot) * (x - knot)^degree
## }
```

Another example: a spline with 2 knots

Start with 3 polynomials

$$p_1(x) = -2x^2 + .5x^3$$

$$p_2(x) = -2x^2 + .5x^3 - 5(x-5)^3$$

$$p_3(x) = -2x^2 + .5x^3 - 5(x-5)^3 + 5(x-6.5)^3$$

Cut them up at $x = 5$ and $x = 6.5$ and tie together with knots there:

$$s(x) = -2x^2 + .5x^3 - 5(x-5)^3 I(x \geq 5) + 5(x-6.5)^3 I(x \geq 6.5)$$

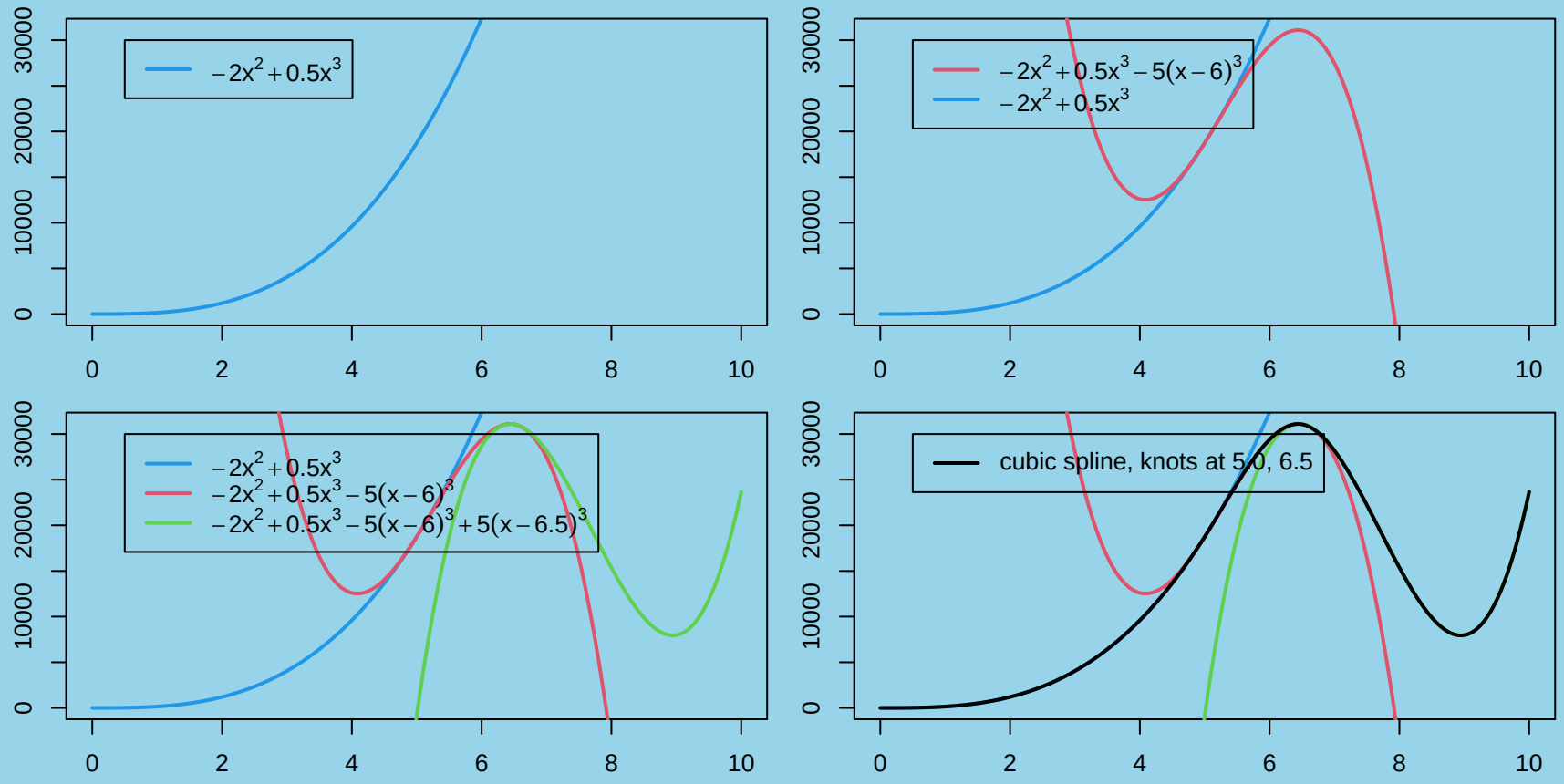
or

$$s(x) = -2x^2 + .5x^3 - 5(x-5)_+^{\,3} + 5(x-6.5)_+^{\,3}$$

This is a cubic spline with knots at 5 and 6.5.

A spline with 2 knots

```
par(mfrow=c(2,2), mar=c(2, 2, 1, 1))
spline.eg2()     # plots three polynomials and a spline
```



Spline regression models:

$$y = \beta_0 + \beta_1 x + \cdots + \beta_k x^k + \gamma_1 T_1(x) + \cdots + \gamma_h T_h(x) + \varepsilon$$

where $T_j(x) = (x - \tau_j)_+^k$.

- **Knots are at** $\tau_1, \tau_2, \ldots, \tau_h$**. These need to be chosen somehow.**
- **The** β**'s and** γ**'s can be estimated by least-squares. The** $\mathbf{X}$ **matrix has** $k + h + 1$ **columns.**
- **All of the estimation, testing and diagnostics of the** `lm()` **function apply.**

A spline model for the transformed windmill data

We consider a quadratic spline with a single knot at $v = 12.5$ ($x = 7.5$):

```
windmill.spl <- lm(out ~ v + I(v^2) + tr.pwr(v, 12.5, 2),
    data = windmill)
```

Let's inspect a few rows of the model matrix $\mathbf{X}$ to see what we have fit:

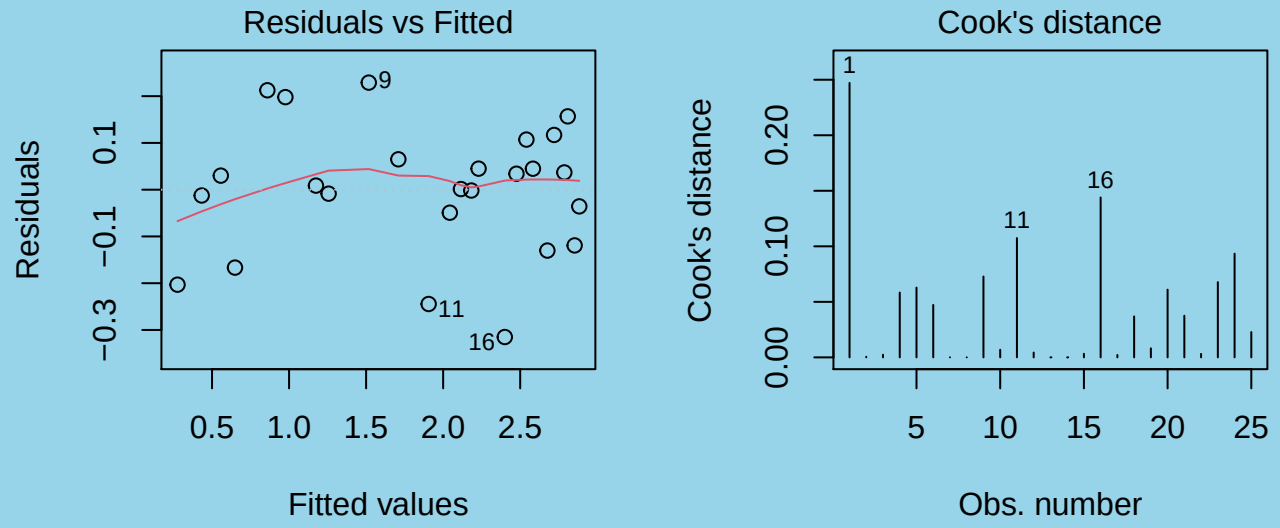
```
model.matrix(windmill.spl)[c(1, 2, 13, 23, 24),]
```

```
##    (Intercept)         v     I(v^2) tr.pwr(v, 12.5, 2)
## 1            1  3.065191   9.395399             0.00000
## 2            1  3.461025  11.978692             0.00000
## 13           1  9.390507  88.181631             0.00000
## 23           1 17.118459 293.041640            21.33016
## 24           1 17.782794 316.227766            27.90791
```

A spline model for the transformed windmill data

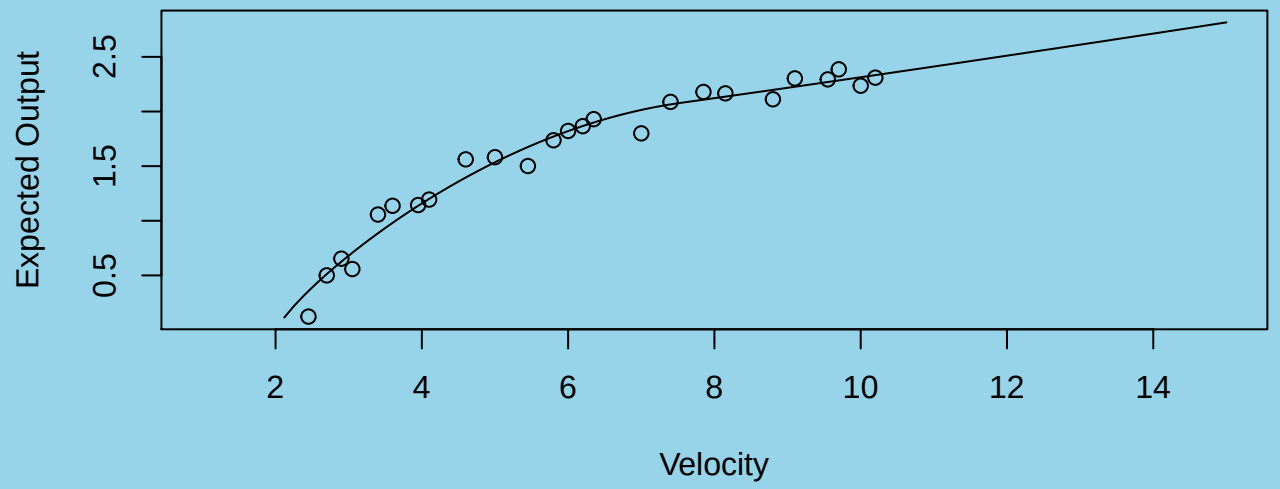
Check the diagnostic residual and influence plots:

```
par(mfrow=c(1, 2))
plot(windmill.spl, which=1); plot(windmill.spl, which = 4)
```



A spline model for the transformed windmill data

```
x <- seq(1, 15, length=101)
plot(x, (predict(windmill.spl,
     newdata = data.frame(v = x^1.25)))^(1/1.25),
     ylab="Expected Output", xlab="Velocity", type="l")
with(windmill, points(Output ~ Velocity))
```

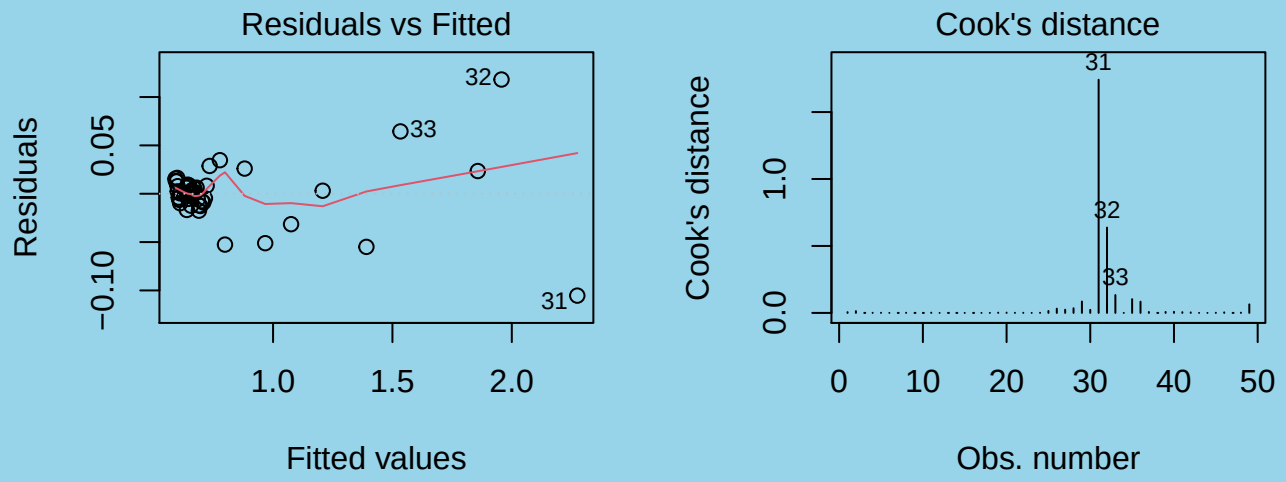


This is somewhat more believable than the earlier cubic polynomial model.

A cubic spline model for the titanium heat data

We consider a cubic spline with 5 knots:

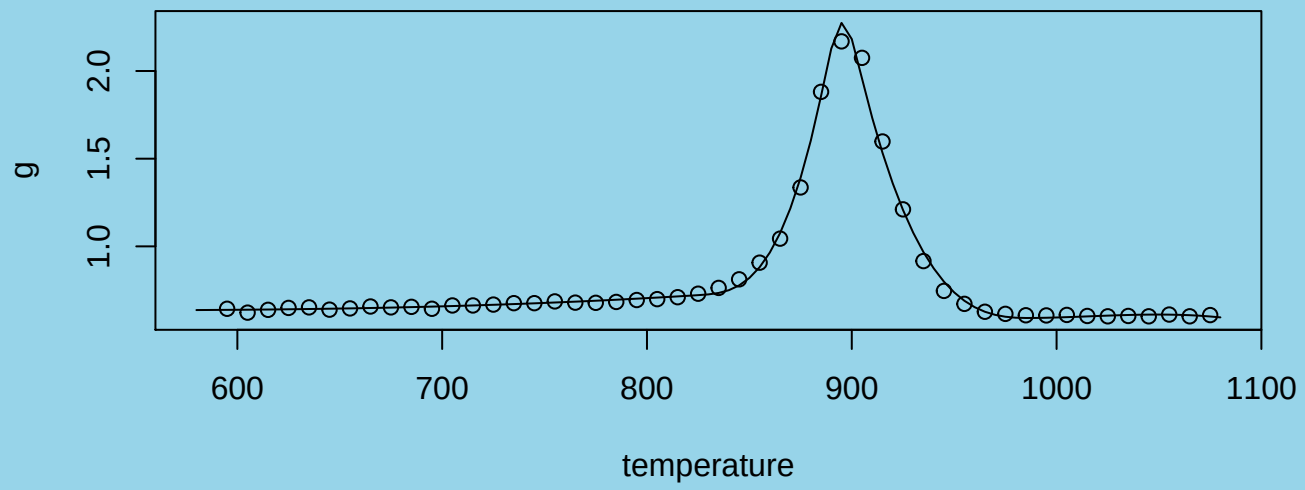
```
titanium$tmp <- titanium$temperature
titan.spl <- lm(g ~ tmp + I(tmp^2) + I(tmp^3) + tr.pwr(tmp, 825, 3) +
tr.pwr(tmp, 885, 3) + tr.pwr(tmp, 895, 3) +  tr.pwr(tmp, 905, 3) +
    tr.pwr(tmp, 990, 3),  data = titanium)
par(mfrow=c(1, 2));  plot(titan.spl, which=1);plot(titan.spl, which=4)
```



There is a hint of trouble at observation 31, but the overall fit is better than the polynomial models.

A cubic spline model for the titanium heat data

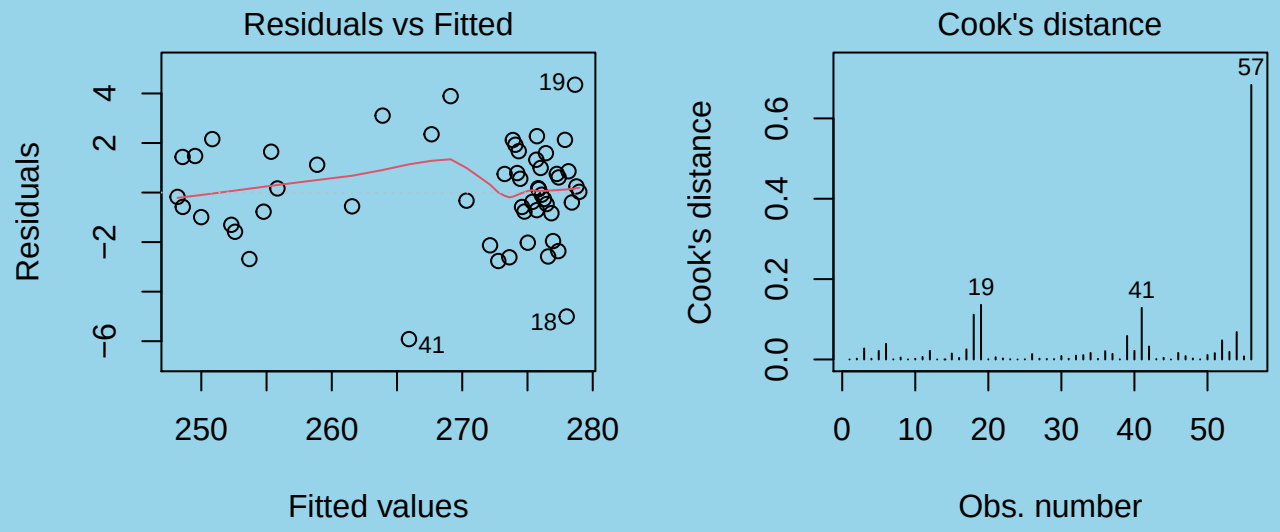
```
x <- seq(580, 1080, length=101)
plot(x, predict(titan.spl,  newdata = data.frame(tmp = x)),
     ylab="g", xlab="temperature", type="l")
with(titanium, points(g ~ temperature))
```



A cubic spline model for the seismic timing data

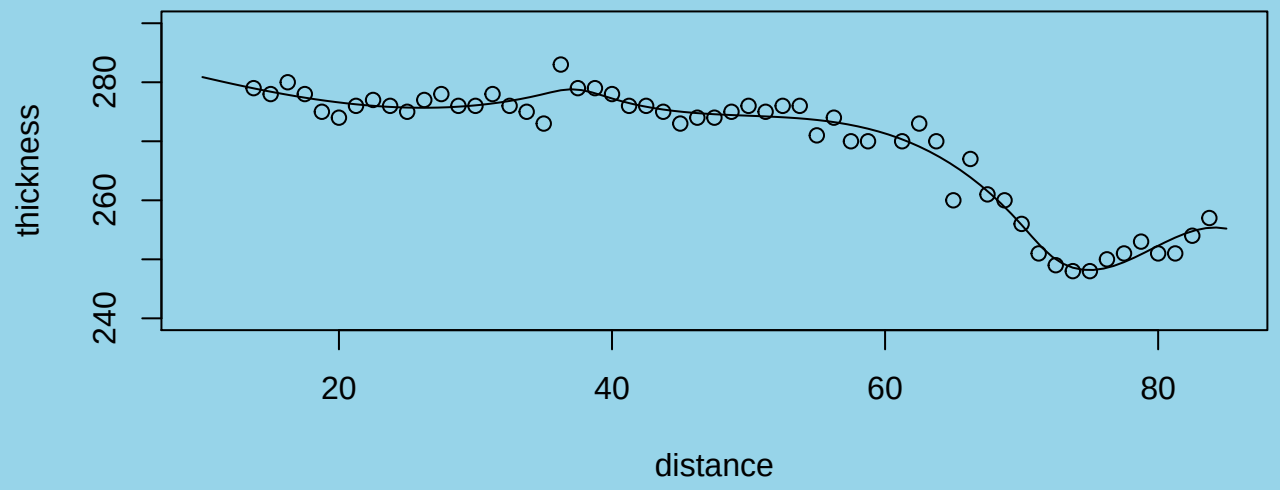
We consider a cubic spline with 5 knots:

```
geo.spl <- lm(thickness ~ distance + I(distance^2) + I(distance^3) +
    tr.pwr(distance, 35, 3) + tr.pwr(distance, 37, 3) +
    tr.pwr(distance, 40, 3) +  tr.pwr(distance, 70, 3) +
    tr.pwr(distance, 72, 3),  data = geophones)
par(mfrow=c(1, 2));  plot(geo.spl, which=1); plot(geo.spl, which = 4)
```



A cubic spline model for the seismic timing data

```
x <- seq(10, 85, length=101)
plot(x, predict(geo.spl,  newdata = data.frame(distance = x)),
     ylab="thickness", xlab="distance", type="l", ylim=c(240, 290))
with(geophones, points(thickness ~ distance))
```

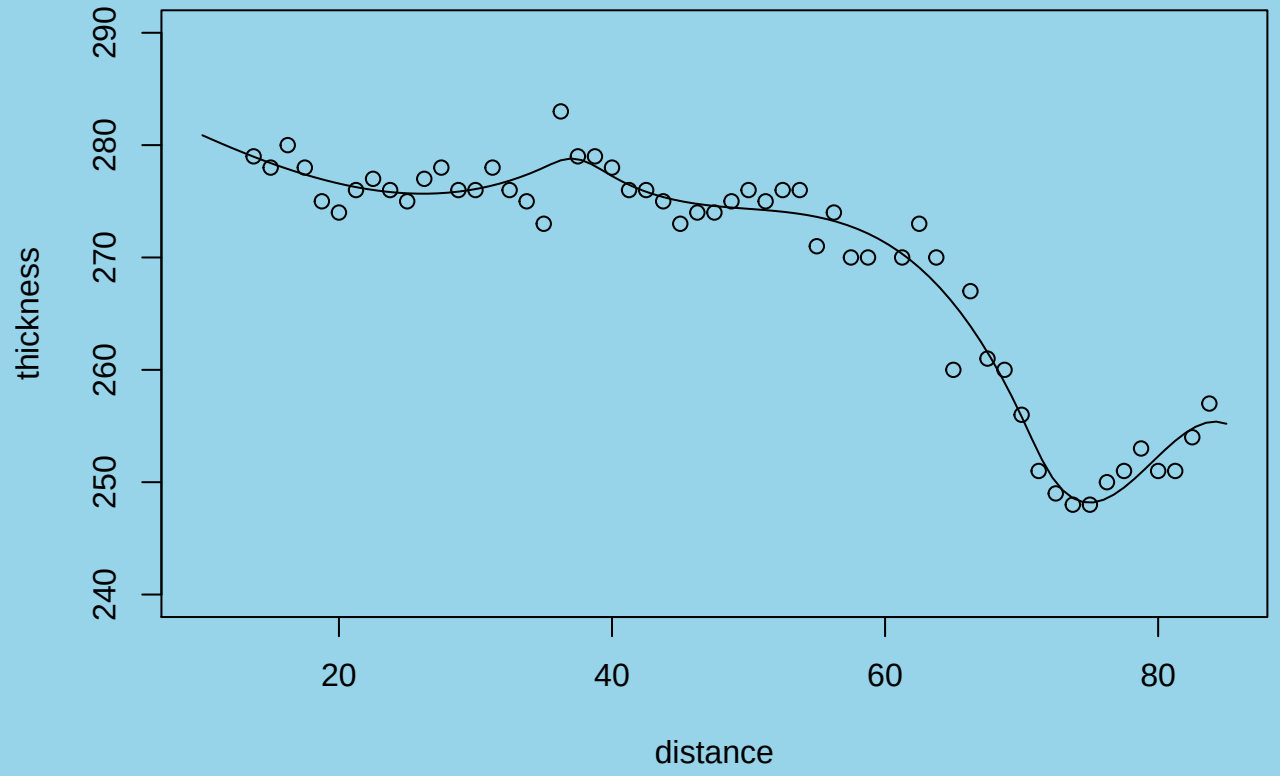


Summary

- **Regression splines offer a parsimonious and more accurate alternative to polynomial regression for data where a linear or quadratic fit is insufficient.**

- **All of the inference machinery for regression (least-squares estimation, variance estimation, confidence intervals, prediction intervals, diagnostic procedures) is available for regression splines.**

Example (cont'd) - a cubic spline model for the seismic timing data



The model matrix for the seismic timing spline model

A few rows of $\mathbf{X}$:

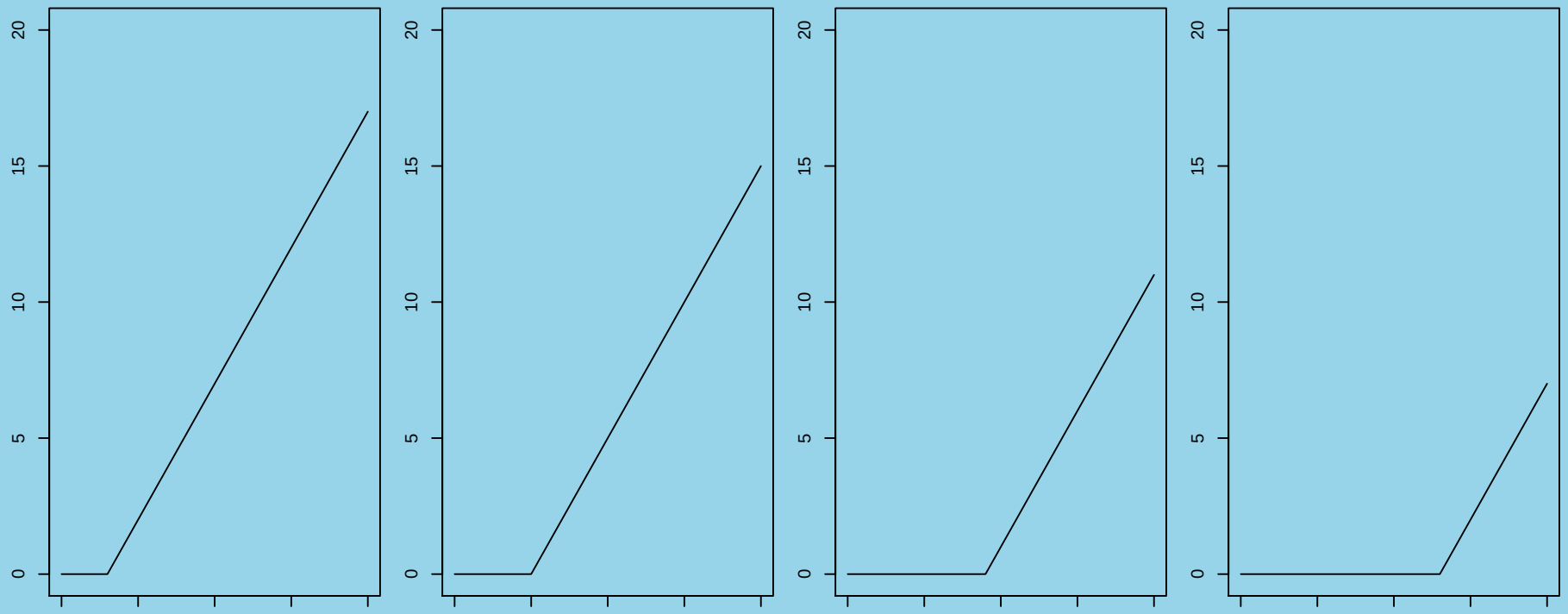
```
X <- model.matrix(geo.spl)
round(X[c(1, 20, 40),], 1)

##    (Intercept) distance I(distance^2) I(distance^3)
## 1            1     13.8         189.1        2599.6
## 20           1     37.5        1406.2       52734.4
## 40           1     63.8        4064.1      259084.0
##    tr.pwr(distance, 35, 3) tr.pwr(distance, 37, 3)
## 1                      0.0                     0.0
## 20                    15.6                     0.1
## 40                 23763.7                 19141.3
##    tr.pwr(distance, 40, 3) tr.pwr(distance, 70, 3)
## 1                      0.0                       0
## 20                     0.0                       0
## 40                 13396.5                       0
##    tr.pwr(distance, 72, 3)
## 1                        0
## 20                       0
## 40                       0
```

When there are many knots, the model matrix has bad numerical properties.. But just as with orthogonal polynomials, there are better ways to transform the x values: B-spline transformations.

Graphing truncated power functions

Consider the degree 1 truncated power functions with knots at 3, 5 and 9 and 13:

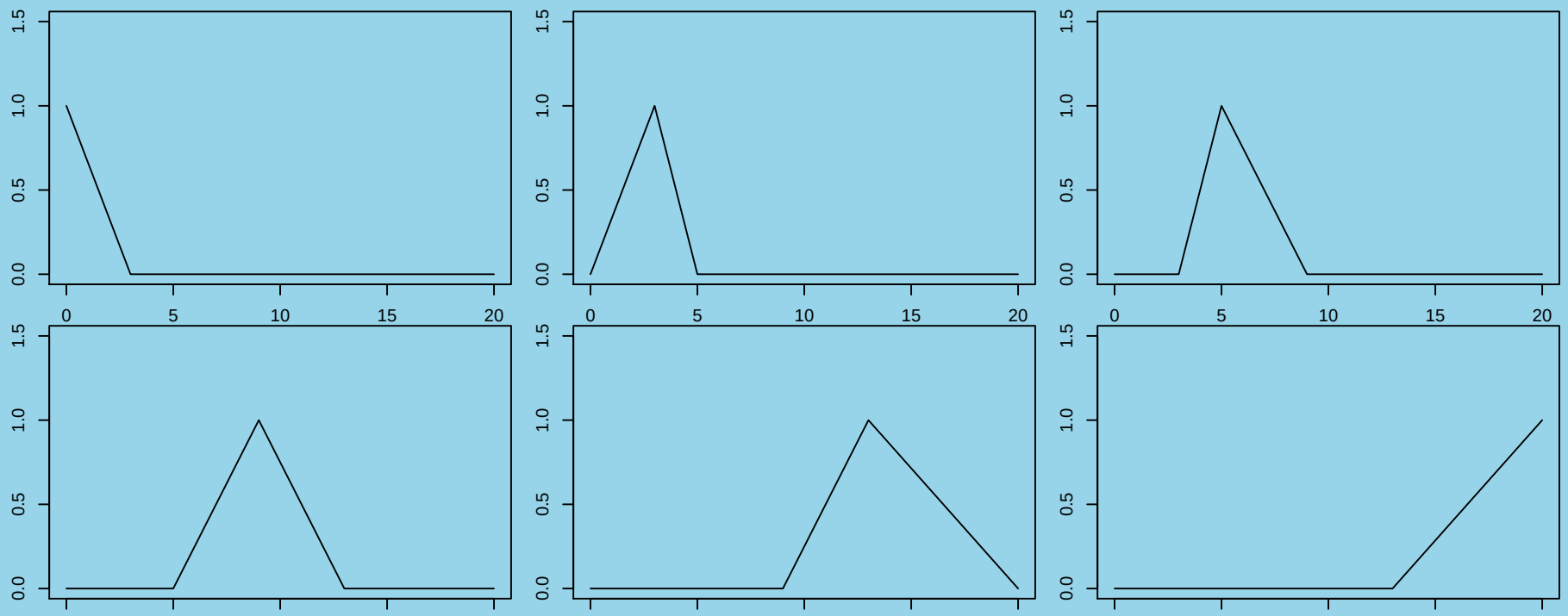
```
par(mfrow=c(1, 4), mar=c(1, 2, 1, 1))
curve(tr.pwr(x, 3, 1), 0,20, ylim=c(0,20));curve(tr.pwr(x, 5, 1), 0,20, ylim=c(0,20))
curve(tr.pwr(x, 9, 1), 0,20, ylim=c(0,20));curve(tr.pwr(x, 13, 1), 0,20, ylim=c(0,20))
```



A linear regression spline with these knots is a linear combination of these functions, together with the constant and linear functions: 6 parameters need to be estimated. i.e. $\mathbf{X}$ is an $n \times 6$ matrix.

Graphing B-spline functions

```
par(mfrow=c(2, 3), mar=c(1, 2, 1, 1)); library(splines)
kt <- c(3, 5, 9, 13); x <- seq(0, 20, length = 101)
for (i in 1:6) plot(bs(x, knots=kt, deg=1, intercept=TRUE)[,i] ~ x,
    type = "l", ylim=c(0,1.5))
```



B-splines

- **B-splines are more numerically stable than truncated splines.**

- **The regression model in terms of B-Splines (`intercept=TRUE` case):**

$$y = \beta_0 B_0(x) + \beta_1 B_1(x) + \beta_2 B_2(x) + \cdots + \beta_k B_k(x) + \varepsilon$$

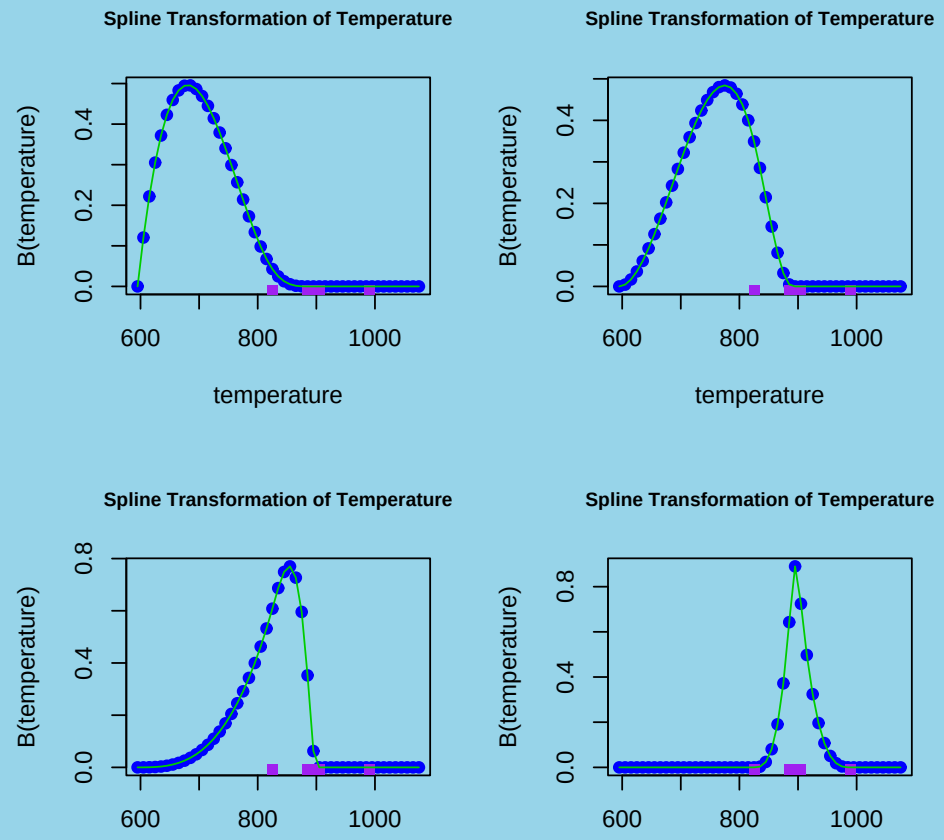
```
lm(y ~ bs(x, deg, knots, intercept=TRUE) - 1)
```

- **The regression model in terms of B-Splines (`intercept=FALSE` case):**

$$y = \beta_0 + \beta_1 B_1(x) + \beta_2 B_2(x) + \cdots + \beta_k B_k(x) + \varepsilon$$

```
lm(y ~ bs(x, deg, knots, intercept=FALSE))
```

The B-spline transformations of temperature (titanium data)



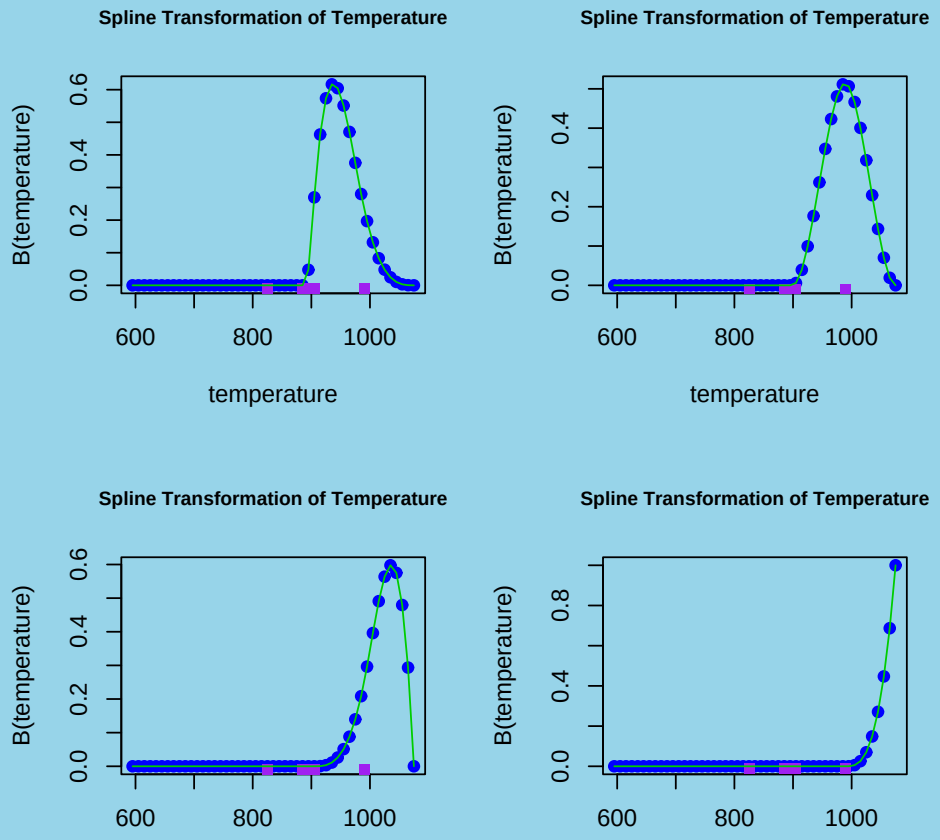
Knots at:
825, 885, 895, 905, 990

Degree = 3

We took `intercept = FALSE` **in the call to** `bs()` **so there are only 8 B-splines, instead of 9.**

The B-spline transformations of temperature (titanium data)

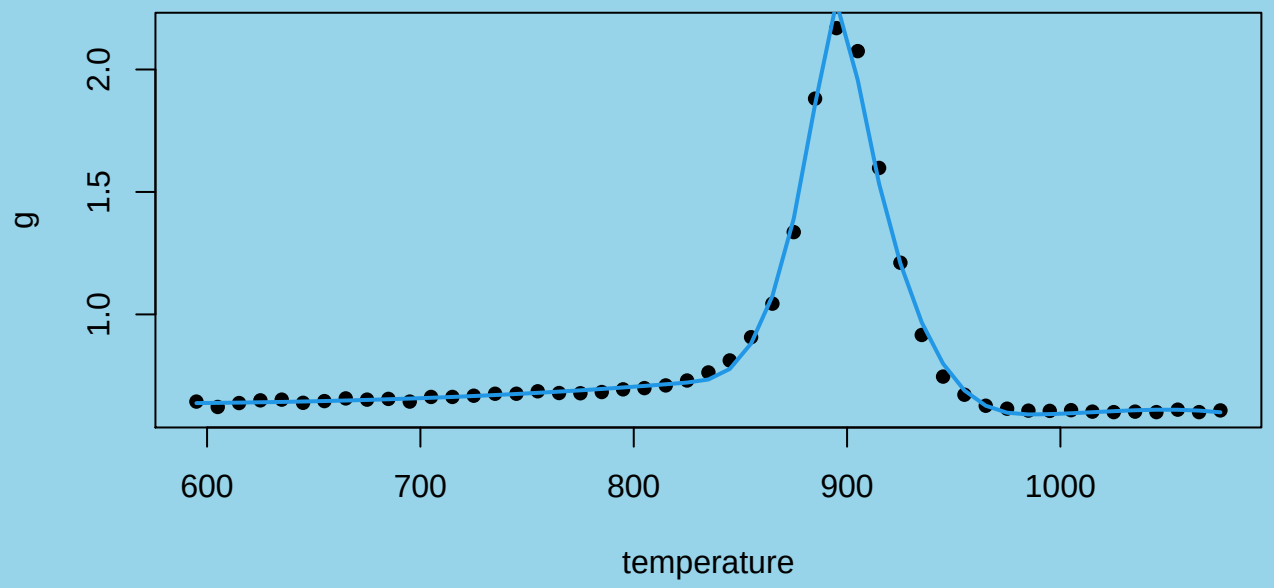
The other four transformations:



Number of transformations = degree + number of knots

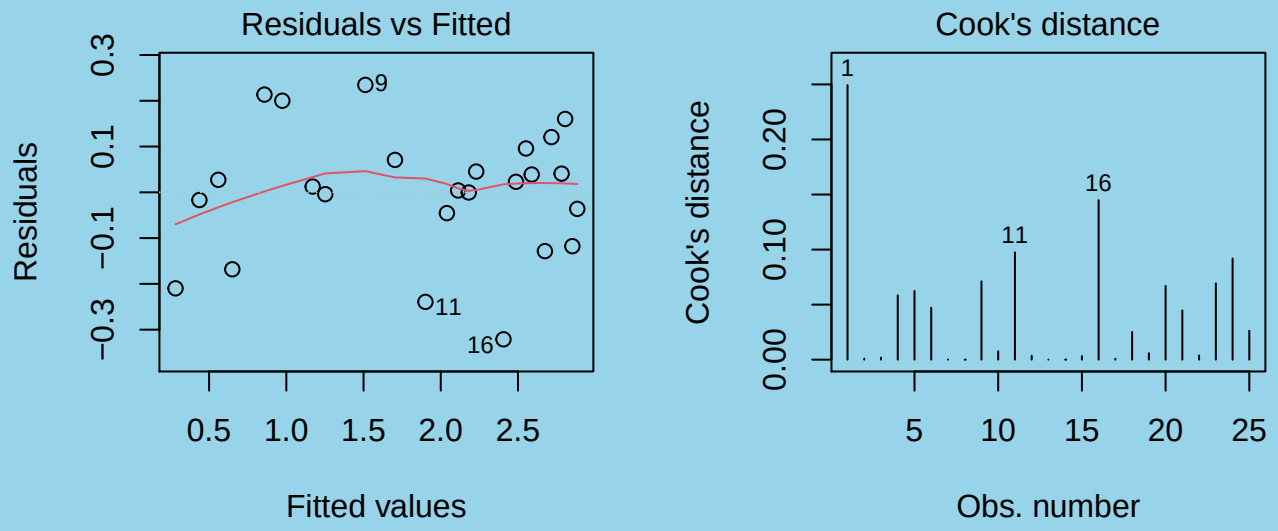
Fitting a cubic spline model to the titanium data

```
titanium.spline<-lm(g ~ bs(temperature,
      knots=c(825,885,895,905,990),degree=3), data = titanium)
plot(g ~ temperature, data = titanium,pch=16)
with(titanium, lines(temperature,predict(titanium.spline), col=4,lwd=2))
```



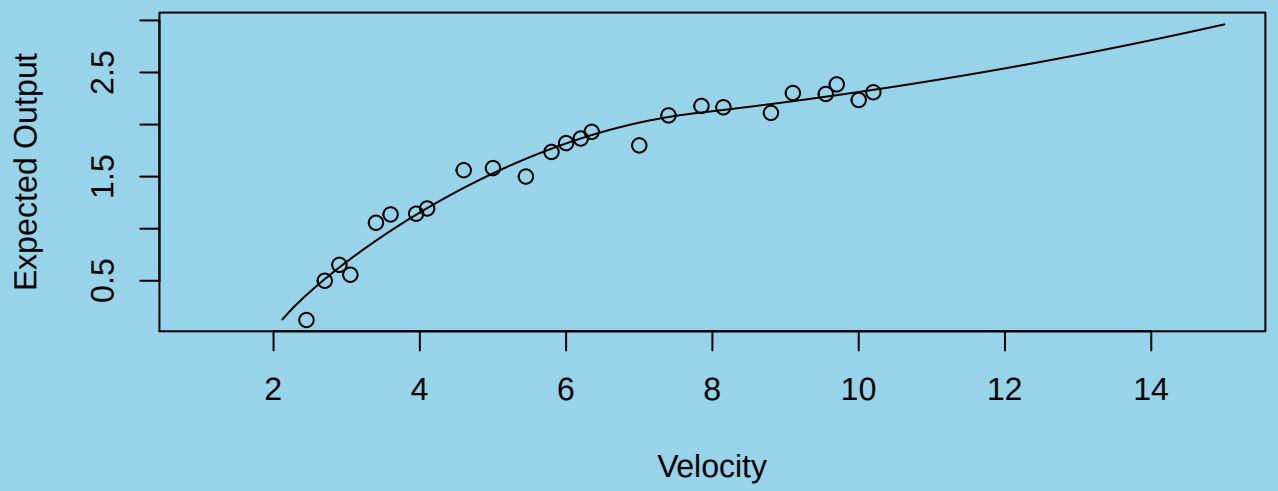
Example – windmill data

```
windmill.lm <- lm(out~ bs(v, degree=2, knots=c(13),
   Boundary.knots=c(0, 35)), data = windmill)
par(mfrow=c(1, 2))
plot(windmill.lm, which=1); plot(windmill.lm, which = 4)
```



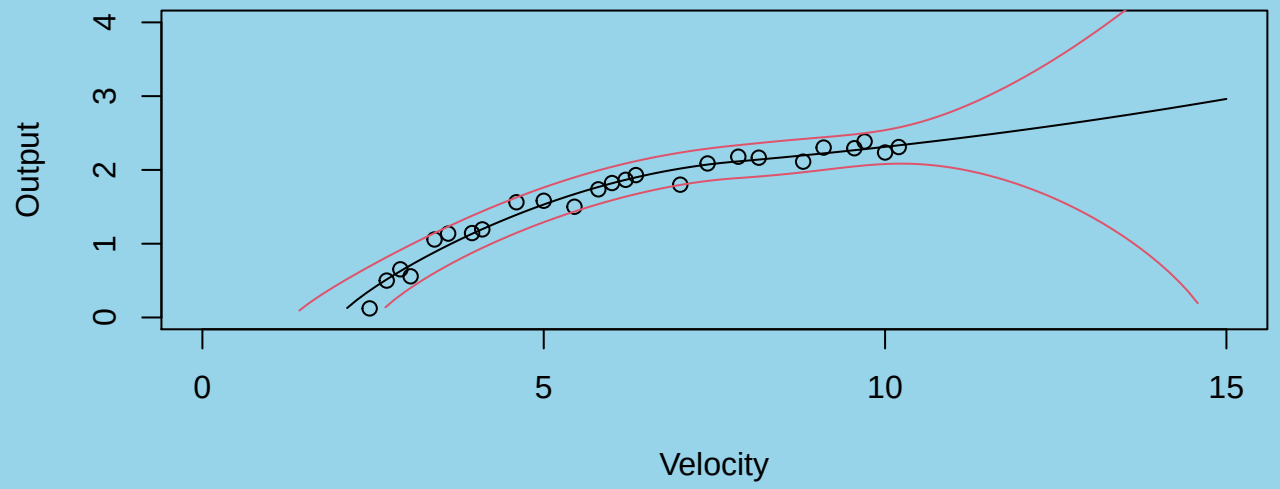
Example – windmill data

```
x <- seq(1, 15, length=101)
plot(x, (predict(windmill.lm, newdata =
  data.frame(v = x^1.25)))^(1/1.25), type="l",
  ylab="Expected Output", xlab="Velocity")
with(windmill, points(Output ~ Velocity))
```



Example – windmill data - prediction Intervals

```
x <- seq(1, 15, length=101)
y <- predict(windmill.lm, newdata =
  data.frame(v = x^1.25), interval="prediction")
plot(Output ~ Velocity, data=windmill, xlim=c(0,15), ylim=c(0,4))
for (j in 1:3) lines(I(y[,j]^(1/1.25)) ~ x, col=1 + (j > 1))
```



The red curves are upper and lower 95% pointwise prediction limits; note how they allow for short range extrapolation, provided the model makes sense.

The problem of knot selection and one work-around

For the geophones data, we selected the knots subjectively.

The smoothing spline allows us to avoid making subjective choices by penalizing the least-squares objective function against large amounts of curvature (i.e. second derivative values).

By using lots of equally spaced knots, we create a highly non-parsimonious model - the number of coefficients to estimate is large relative to n. We regularize this problem by adding a quadratic penalty on these coefficients.

There is a penalty parameter λ which can be chosen by leave-one-out cross-validation.

Smoothing splines are the earliest example of a larger class of penalized splines.

A smoothing spline model for the seismic timing data

```
geo.ss <- with(geophones, smooth.spline(distance, thickness))
plot(geophones)
lines(geo.ss)
```